



兰州工业学院

实 验 报 告

(2024—2025学年第1学期)

课程名称 嵌入式系统应用

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验五 串行端口程序设计		
实验时间	2024. 10. 28	实验地点	信息与通信工程专业实验室（嵌入式系统平台）
一、实验内容 1. 了解在linux环境下串行程序设计的基本方法 2. 掌握终端的主要属性及设置方法，熟悉终端I/O函数的使用 3. 学习使用多线程来完成串口的收发处理 4. 学习将多线程编程应用到串口的接收和发送程序设计中 5. 编写应用程序实现对 ARM 设备串口的读和写 二、实验器材（设备、元器件） 1. 个人计算机一台 2. Ubuntu Linux操作系统 3. IMX6嵌入式教学科研平台 三、实验步骤 1. 描述本次串行端口通信程序设计的功能和思路 初始化串口：首先，程序需要初始化串口，设置合适的波特率、数据位、停止位和校验位等参数，以确保数据传输的正确性。 创建线程：程序使用多线程技术，创建两个线程分别用于串口数据的接收和发送。这样可以同时进行数据的接收和发送，提高程序的效率。 串口接收线程：接收线程负责从串口读取数据。当串口接收到数据时，线程会读取这些数据，并将其打印到控制台。为了确保串口写操作的原子性，接收到的数据在写回串口之前会使用互斥锁进行同步。 串口发送线程：发送线程负责向串口发送数据。程序设计了一个简单的循环，发送线程会周期性地向串口发送字符，并在每次发送后休眠一定时间，以模拟实际的数据发送过程。 互斥锁的使用：为了避免多个线程同时访问串口导致数据混乱，程序在发送和接收数据时使用了互斥锁。这确保了在任何时候只有一个线程能够对串口进行操作。 程序同步：使用 pthread_join 函数等待接收和发送线程结束，确保主程序			

在子线程完成工作后才继续执行，最后关闭串口并退出。

错误处理：程序中包含了对串口打开失败等错误情况的处理，确保了程序的健壮性。

2. 绘制串行端口通信程序设计的流程图



图 一 主程序流程图

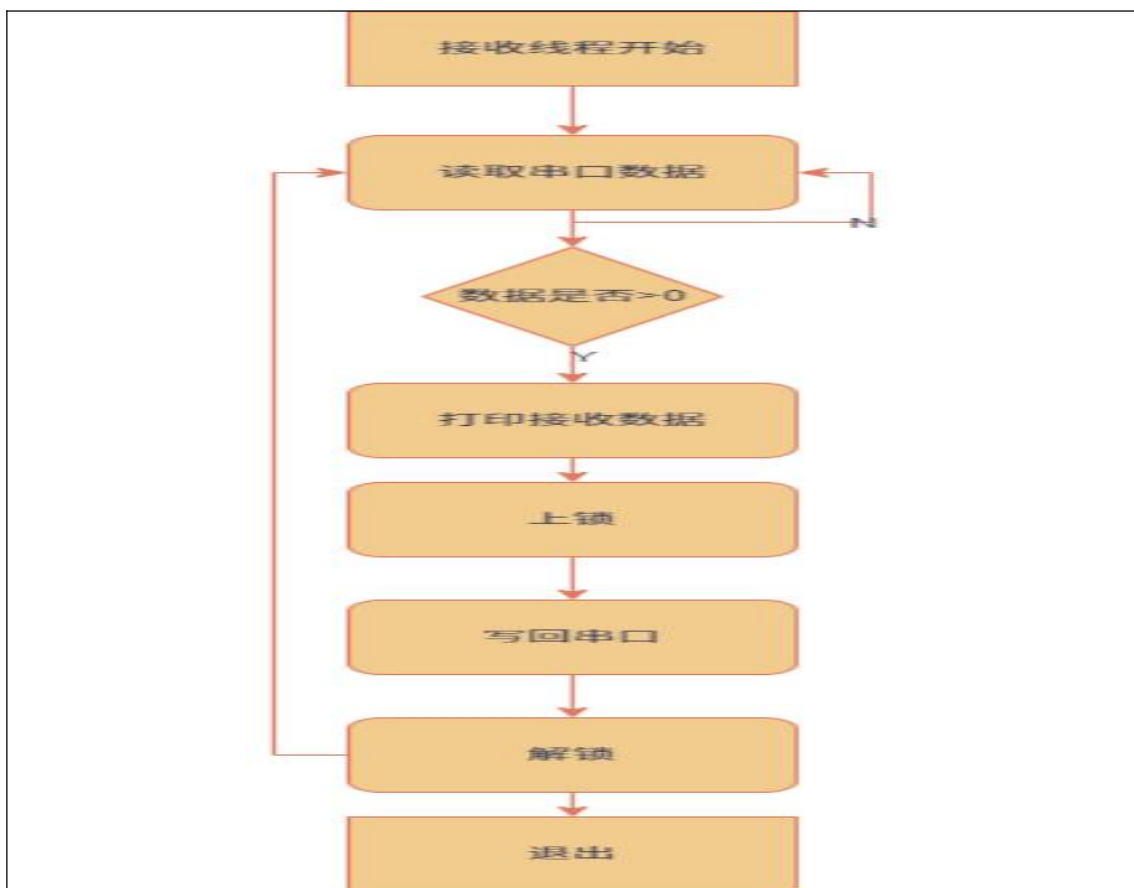


图 二 接收线程流程图

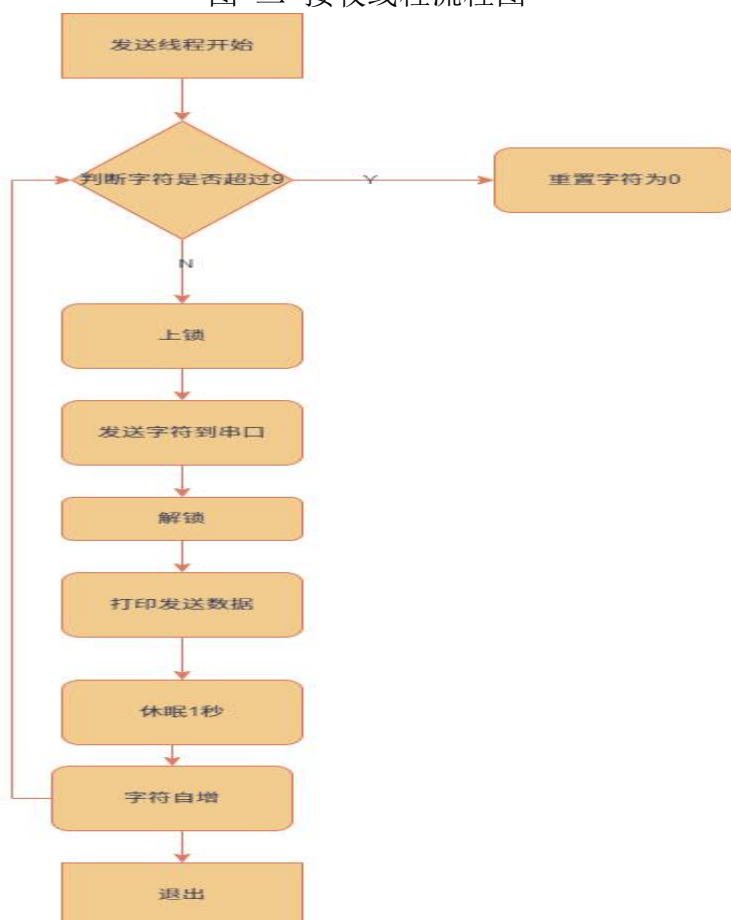


图 三 发送线程流程图

3. 在Ubuntu开发程序实现代码编译

```
term.c x
#include <termios.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/types.h>
#include <string.h>
#include <sys/signal.h>
#include <pthread.h>

#define BAUDRATE    B115200 // 定义波特率
#define COM         "/dev/ttyS1" // 定义串口设备文件路径
#define FALSE       0
#define TRUE        1

static int fd; // 串口文件描述符
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // 初始化互斥锁

// 初始化串口通信
void com_init(speed_t speed)
{
    struct termios options; // 串口配置结构体
    fd = open(COM, O_RDWR | O_NOCTTY | O_NDELAY); // 以读写方式打开串口，不控制终端，非阻塞模式
    if (fd < 0)
    {
        printf("打开串口设备失败\n");
        exit(1); // 如果打开串口失败，则退出程序
    }
    tcgetattr(fd, &options); // 获取当前串口配置
    cfsetispeed(&options, speed); // 设置输入波特率
}
```

图 四 编写代码

```
root@jy:/home/IMX6/exp/basic/03_tty# gcc term.c -o term -lpthread
root@jy:/home/IMX6/exp/basic/03_tty#
```

图 五 代码编译

4. 启动IMX6嵌入式实验平台使用NFS共享目录挂载运行测试

```
root@jy:/home/jy/桌面# mount -t nfs 192.168.16.2:/home/IMX6/ /mnt/
```

图 六 挂载

四、实验数据及结果分析

1. 交叉编译源代码

```
#include <termios.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/types.h>
#include <string.h>
#include <sys/signal.h>
#include <pthread.h>
```

```

#define BAUDRATE      B115200 // 定义波特率

#define COM           "/dev/ttyS1" // 定义串口设备文件路径

#define FALSE         0

#define TRUE          1

static int fd; // 串口文件描述符

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // 初始化互斥锁

// 初始化串口通信

void com_init(speed_t speed)
{
    struct termios options; // 串口配置结构体

    fd = open(COM, O_RDWR | O_NOCTTY | O_NDELAY); // 以读写方式打开串
    口，不控制终端，非阻塞模式

    if(fd < 0)
    {
        printf("打开串口设备失败\n");
        exit(1); // 如果打开串口失败，则退出程序
    }

    tcgetattr(fd, &options); // 获取当前串口配置

    cfsetispeed(&options, speed); // 设置输入波特率

    cfsetospeed(&options, speed); // 设置输出波特率

    options.c_cflag |= (CLOCAL | CREAD); // 启用串口，忽略 modem 控制
    线

    options.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG); // 设置为非规
    范输入，不 echo，不处理特殊字符

    options.c_oflag &= ~OPOST; // 输出处理关闭

    options.c_iflag &= ~(BRKINT | ICRNL | INPCK | ISTRIP | IXON); //
    输入处理关闭

```

```

    tcsetattr(fd, TCSANOW, &options); // 应用配置
}

// 串口接收数据处理
void* receive(void * data)
{
    char ch[1024]; // 用于存储接收到的数据
    int ret;
    printf("开始读取串口数据\n");
    while (1)
    {
        ret = read(fd, ch, sizeof(ch) - 1); // 从串口读取数据
        if(ret > 0)
        {
            ch[ret] = '\0'; // 确保字符串以 NULL 结尾
            printf("接收到的数据: %s\n", ch); // 打印接收到的数据
            pthread_mutex_lock(&mutex); // 上锁，确保串口写操作的原子
性
            write(fd, ch, ret); // 将接收到的数据写回串口
            pthread_mutex_unlock(&mutex); // 解锁
        }
    }
    printf("\n");
    printf("退出串口读取\n");
    return NULL;
}

// 串口发送数据
void* send(void * data)
{

```

```

    int c = '0';

    printf("开始发送数据\n");

    while (1)
    {
        if(c > '9')
        {
            c = '0'; // 如果发送的字符超过'9'，则重置为'0'
        }

        pthread_mutex_lock(&mutex); // 上锁，确保串口写操作的原子性
        write(fd, &c, 1); // 向串口发送一个字符
        pthread_mutex_unlock(&mutex); // 解锁
        printf("发送的数据: %c\n", c); // 打印发送的数据
        sleep(1); // 休眠 1 秒
        c++; // 发送下一个字符
    }

    printf("\n");

    return NULL; // 子线程结束，返回 NULL
}

// 主函数
int main(int argc, char** argv)
{
    pthread_t th_a, th_b; // 线程 ID
    void * retval;

    pthread_mutex_init(&mutex, NULL); // 初始化互斥锁
    com_init(BAUDRATE); // 初始化串口

    pthread_create(&th_a, NULL, receive, NULL); // 创建接收线程
    pthread_create(&th_b, NULL, send, NULL); // 创建发送线程

```



```

// 等待子线程结束

pthread_join(th_a, &retval);

pthread_join(th_b, &retval);

close(fd); // 关闭串口

exit(0); // 程序正常退出
}

```

2. 在ARM嵌入式Linux平台执行程序验证结果和分析串口数据输入输出的过程

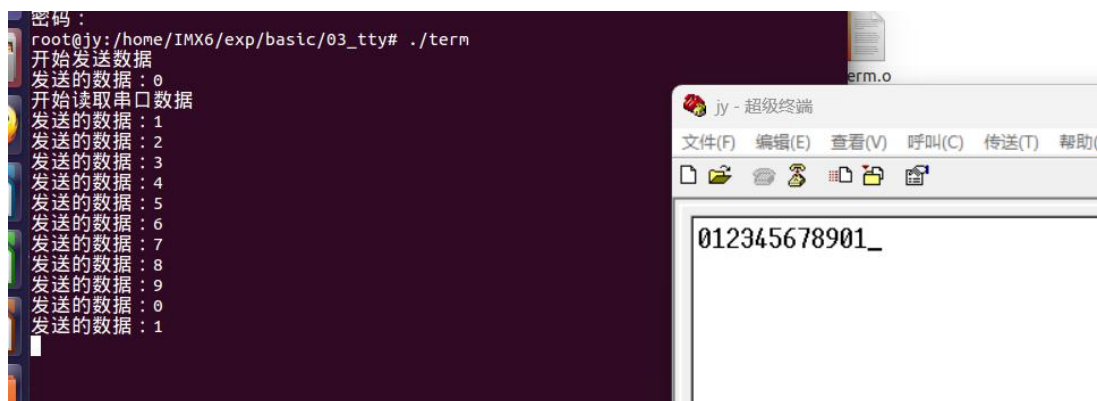


图 七 运行结果

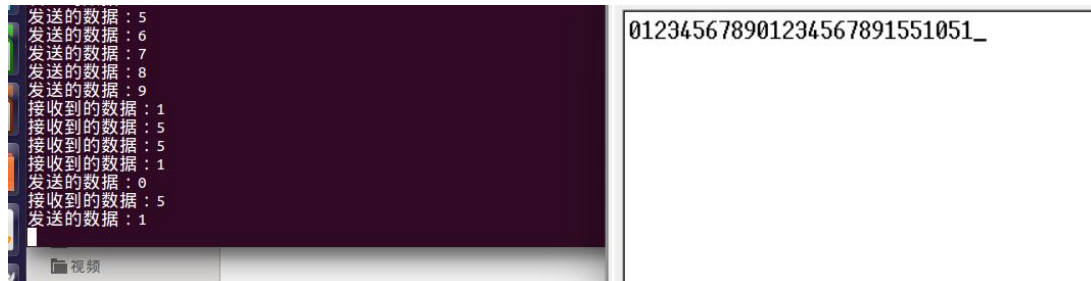


图 八 在超级终端输入测试结果

五、总结

实验成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与表现	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			刘建明 张翰茹

备注：考核项目及分值分布应依据教学大纲确定。